Devoid Viewer

Codebase & Installer Documentation

This document explains every file, struct, method, and design decision in the Devoid Viewer codebase, and documents the Inno Setup installer. Written for someone new to Rust who wants to understand not only *what* the code does but *why* it was written that way.

Table of Contents

Section	Topic	Page
1	Project structure overview	2
2	Cargo.toml — dependencies explained	2
3	nav.rs — directory navigation	4
4	loader.rs — image decoding	7
5	main.rs — application core	11
6	Threading & asynchronous design	24
7	Key design decisions & trade-offs	27
8	Caveats & known limitations	30
9	Installer — devoid_viewer_setup.iss	30
10	Glossary of Rust concepts used	33

1. Project Structure Overview

File	Responsibility
`Cargo.toml`	Project name, version, and all external dependencies.
`build.rs`	Embeds devoid.ico and version metadata into the exe via winres.
`src/main.rs`	Event loop, app state, input handling, pixel renderer, window management, directory watching.
`src/loader.rs`	Image decoding for all supported formats. Owns Frame and DecodedImage.
`src/nav.rs`	Directory scanning, human-sorted sibling list, next/prev navigation.
`devoid_viewer_setup.iss`	Inno Setup 6 installer script.
`devoid.ico`	Multi-size application icon. Embedded in the exe at compile time.

Supported formats: BMP, PNG, JPEG, GIF (animated), WebP, TIFF, ICO, JPEG XL. SVG was intentionally removed to keep the binary ultra-lightweight.

2. Cargo.toml — Dependencies Explained

Each crate was chosen for leanness. The dependency graph is roughly 50–70 crates and the release binary around 2–3 MB.

Windowing: winit

winit handles the event loop, keyboard, window resize, drag-and-drop, and monitor queries. The rwh_05 feature enables the raw-window-handle v0.5 interface that softbuffer requires. On Windows the platform extension EventLoopBuilderExtWindows installs a Win32 message hook.

Pixel display: softbuffer

softbuffer provides a Vec<u32> that maps directly to the window's pixels. No GPU pipeline, no shader compilation. On Windows it uses a GDI DIB section accelerated by the compositor.

> ■ *DWM fills every new window with white before the application paints. Mitigated by hiding the window until the image is ready and calling paint_background() synchronously before the event loop starts.*

Native dialogs: windows-sys

windows-sys is a zero-overhead Windows API binding. Used for MessageBoxW (all dialogs), SystemParametersInfoW/SPI_GETWORKAREA (precise taskbar exclusion), and PostMessageW (taskbar thumbnail close fix).

Image decoding: image

Handles BMP, PNG, JPEG, GIF, WebP, TIFF, and ICO. GIF animation uses GifDecoder with AnimationDecoder. ICO is also used to decode devoid.ico for the window icon.

JPEG XL: jxl-oxide

Pure-Rust JPEG XL decoder. Uses the low-level streaming API.

Filesystem watching: notify-debouncer-mini

notify-debouncer-mini wraps the platform-native filesystem watching APIs (ReadDirectoryChangesW on Windows, FSEvents on macOS, inotify on Linux) and adds a configurable debounce window. default-features = false drops optional backends not needed here. Zero steady-state CPU cost — the OS delivers events only when the directory actually changes.

Channel communication: crossbeam-channel

Faster than std::sync::mpsc. Bounded at capacity 32 to throttle background threads if the UI falls behind.

Windows resource embedding: winres (build dependency)

Runs at build time only. Embeds devoid.ico and version metadata. Zero runtime cost.

3. nav.rs — Directory Navigation

Maintains a human-sorted list of sibling image files in the same directory as the currently open file. Navigation wraps at both ends.

struct DirNavigator — fields

- **files: Vec<PathBuf>** — human-sorted sibling list.
- **index: usize** — position of the currently displayed file.
- **current_dir: Option<PathBuf>** — the directory files was built from. Stored explicitly to avoid inferring it from files.first(), which gives wrong answers after the single-entry fallback resets the list.

fn set_current(path)

Compares the new path's parent against current_dir. Rebuilds only on change. After rebuilding, locates path by position(). The fallback (single-entry list) handles files in archiver temp folders.

fn next() / fn prev() / fn peek_next() / fn peek_prev()

All delegate to the private step() and peek() helpers via the Direction enum, eliminating duplication. Return None for single-image directories.

fn remove(path)

Removes a path after deletion. Uses saturating_sub to keep the index in bounds.

fn rename(old_path, new_path)

Replaces a path after a successful rename. Preserves the current index.

fn len() / fn contains(path)

len() returns the number of images in the current sibling list. Used by navigate() as the loop bound to prevent infinite cycling when all files are externally deleted. **contains()** checks whether a specific path is still in the list — used by on_directory_changed() to detect external deletion.

fn rebuild_current(current_path)

Rebuilds the sibling list then re-locates current_path by position() in the new sorted list so the index stays accurate regardless of how many files were added or removed. If the current file is absent, the index is clamped. Takes current_path as a parameter because apply_image() calls set_current() before this method runs, making the navigator's own state unreliable as a source.

fn rebuild() — private

Populates files from current_dir. Uses DirEntry::file_type() — available from the readdir() result at no extra syscall cost. is_file() would issue one extra stat() per file.

enum Direction / fn step / fn peek / fn wrapped_index

Direction::Forward and Direction::Backward eliminate the bool parameter duplication between next/prev and peek_next/peek_prev. wrapped_index() computes the neighbour index with wraparound.

fn human_sort / fn consume_number

Splits filenames into text and digit runs. Digit runs are compared numerically so img2.png sorts before img10.png. c.to_digit(10) extracts digits cleanly. Saturating arithmetic prevents overflow on pathological inputs.

4. loader.rs — Image Decoding

Owns the image type definitions and all decoding logic. Returns DecodedImage directly — no conversion needed in main.rs. No knowledge of UI state.

struct Frame / struct DecodedImage

Frame: straight RGBA8 pixels, row-major, plus delay_ms (0 for non-animated). **DecodedImage**: one or more frames sharing canvas dimensions. frame(idx) has a debug_assert for bounds checking. single() is the private constructor for single-frame images.

enum LoadOutcome

- **Ok(DecodedImage)** — decoded using the declared extension.
- **WrongExtension(DecodedImage, String)** — decoded via header sniff; the String is the correct extension. Image displayed immediately; rename dialog offered after the drain loop.
- **Unsupported(String)** — not recognised even after sniffing.

fn load_image — two-pass strategy

Fast path: decode_by_ext with ByExtension. This validates magic bytes against the declared extension — if they disagree, fails immediately. On failure: sniff_format reads 32 bytes. If a different format is detected, decode with ByContent and return WrongExtension. If the same extension is sniffed but decode still fails, the file is corrupt.

enum DecodeStrategy / fn decode_by_ext

ByExtension validates magic bytes then calls image::open(). ByContent uses with_guessed_format() ignoring the extension. decode_by_ext dispatches to decode_jxl, decode_gif, or decode_raster based on extension.

■ *image::open() and with_guessed_format() both sniff the header and succeed silently on misnamed files. The explicit magic check in ByExtension is the only reliable way to detect extension mismatches.*

fn magic_to_format — single source of truth

Identifies format from first 12 bytes. Used by both sniff_format and decode_raster to ensure consistency.

Format	Magic bytes	Bytes needed
JPEG	FF D8	2
PNG	89 50 4E 47	4
BMP	42 4D	2
GIF	47 49 46 38 (GIF8)	4
TIFF	49 49 2A 00 or 4D 4D 00 2A	4

ICO	00 00 01 00	4
WebP	52 49 46 46 ?? ?? ?? ?? 57 45 42 50	12

fn format_to_ext / fn sniff_format / fn read_magic

format_to_ext maps ImageFormat to extension string. **sniff_format** reads 32 bytes, delegates to magic_to_format via format_to_ext, then checks JXL signatures separately. **read_magic** reads 12 bytes and returns the detected ImageFormat — used by decode_raster for the ByExtension validation.

fn decode_raster / fn decode_gif / fn decode_jxl

- **decode_raster ByExtension**: derives declared format from extension, calls read_magic() for actual format, returns Err on mismatch, then image::open().
- **decode_raster ByContent**: with_guessed_format() → decode().
- **decode_gif**: GifDecoder + AnimationDecoder. Corrupt frames are skipped. 20ms minimum delay; 100ms fallback for zero denominator.
- **decode_jxl**: sRGB output, 32768px dimension guard, checked_mul overflow guard, delegates conversion to jxl_samples_to_rgba8.

fn jxl_samples_to_rgba8

Converts 1–4 channel f32 samples to RGBA8. Greyscale broadcast to RGB; missing alpha defaults to opaque. Extracted from decode_jxl for clarity.

5. main.rs — Application Core

5.1 Type aliases and constants

- **DirWatcher** — type alias for the concrete debouncer type, keeping field declarations readable.
- **Proxy** — type alias for EventLoopProxy<WakeUp>.
- **BG, APP_NAME, DOUBLE_CLICK** — background grey, app name, double-click window.
- **DIR_WATCH_DEBOUNCE: Duration = 50ms** — debounce window for the directory watcher. Named constant so it is visible and changeable in one place.

5.2 enum Msg

- **Loaded / WrongExtension / Preloaded / Unsupported / Deleted** — as before.
- **DirectoryChanged** — zero-payload variant sent by the watcher thread when the OS reports a file addition or deletion in the watched directory. No path information is needed because rebuild_current() re-scans everything.

5.3 struct App — directory watching fields

- **dir_watcher: Option<DirWatcher>** — the live watcher handle. Dropping it unregisters the OS watch. Replaced when the directory changes.
- **watched_dir: Option<PathBuf>** — the directory currently being watched. Tracked separately from the navigator because open_image() calls set_current() (updating the navigator) before the decode thread finishes and apply_image() runs. Using the navigator to detect directory changes would always see the same directory and never install the watcher.

5.4 fn apply_image — watcher installation

Compares new_dir (the parent of the incoming path) against watched_dir. On first call watched_dir is None, so the check detects a change and calls watch_dir(). On same-directory navigation watched_dir already matches, so the existing watcher is kept. On cross-directory navigation the old watcher is replaced. navigator.set_current() is called after the watcher check so the watcher decision is based on watched_dir, not the navigator.

5.5 fn watch_dir

Creates a Debouncer with DIR_WATCH_DEBOUNCE (50ms). The closure sends Msg::DirectoryChanged + WakeUp on any successful event result. Uses an immediately-invoked closure returning Option so the two fallible steps (new_debouncer and watch()) can use ? and the result is assigned cleanly. On any failure dir_watcher and watched_dir are both set to None — navigation degrades gracefully to a static sibling list.

5.6 fn on_directory_changed

Clears the preload cache (entries may reference deleted files), calls navigator.rebuild_current(current_path) to re-scan and re-locate the current file, then checks navigator.contains() to see if the current file survived. If not, navigates to the next available sibling or

clears the display.

5.7 fn navigate — external deletion safety

Loops up to navigator.len() times. On each step, checks path.exists() before calling open_image(). If the path is gone, calls navigator.remove() and preload_cache.remove() and tries the next sibling. This handles the race where the user navigates before the OS watch event arrives. If all siblings are gone, clears the preload cache and calls clear_image().

5.8 fn drain_messages — DirectoryChanged arm

Calls on_directory_changed(proxy) when Msg::DirectoryChanged is received. Runs inside the non-blocking try_recv loop like all other messages.

5.9 fn toggle_maximize

Saves window.inner_size() (physical pixels). On restore, calls request_inner_size(size) and sets pending_center = true. No-op in fullscreen. debug_assert verifies pre_maximize is Some whenever maximized is true.

5.10 fn open_image

Cache hit: apply_image directly. Cache miss: seeds the navigator via set_current (so peek_next/peek_prev are accurate for preloading), then spawns a background thread. send() failures are ignored — they only occur when the receiver is dropped (process exiting).

5.11 fn preload_adjacent

Speculatively decodes both neighbours. Only caches LoadOutcome::Ok — wrong-extension files must not be silently preloaded as they would skip the rename dialog.

5.12 fn render — row-based optimisation

- Rows entirely above or below the image are filled with row.fill(bg) — one slice call rather than per-pixel branching.
- Left and right margins are filled with slice.fill(bg).
- inv_scale = 1.0 / scale computed once per frame, multiplied per pixel.
- composite() fast-paths a==255 and a==0 to skip the blend calculation.

5.13 fn offer_rename — writability probe

Opens the file with write access to test writability. On Windows, all shell folders (Desktop, Downloads, Pictures) have the read-only attribute set by convention for Explorer's custom-view mechanism — checking the attribute would suppress the rename dialog for every file in those folders. Checking the file itself is reliable regardless of folder.

5.14 Window helpers

- **work_area_rect()** — Windows SPI_GETWORKAREA as (l,t,r,b) physical pixels.
- **current_or_any_monitor()** — prefers current monitor, falls back to any.
- **heuristic_work_area()** — 90%×85% of monitor, anchored at its origin.
- **clamp_to_work_area()** — measures title bar from live window frame.
- **center_window()** — uses work_area_rect() on Windows, clamps bottom.

6. Threading & Asynchronous Design

6.1 The channel pipeline

- One bounded crossbeam channel, capacity 32. msg_tx cloned per thread.
- msg_rx drained non-blockingly on every WakeUp.
- Background threads post Msg then call proxy.send_event(WakeUp).
- The directory watcher also posts Msg::DirectoryChanged + WakeUp via the same channel when the OS delivers a filesystem event.

6.2 Directory watching — design

notify-debouncer-mini wraps the OS-native watching mechanism and coalesces rapid bursts into a single callback after a 50ms quiet period. The watcher runs on its own internal thread (created by the crate). The callback sends Msg::DirectoryChanged to the UI channel and fires a WakeUp, exactly like decode threads. Zero idle CPU — the OS delivers events only on change.

6.3 The watched_dir / navigator separation

open_image() calls navigator.set_current() before the decode thread starts. apply_image() runs later when the decode thread's result arrives. By that time, navigator.current_dir() already reflects the new directory — so comparing against it would always show 'same directory' and never install the watcher. watched_dir is updated only inside watch_dir(), making it the reliable signal for whether a new watcher is needed.

6.4 External deletion safety — two layers

Layer 1 (proactive): The watcher fires when a file is deleted, triggering on_directory_changed() which rebuilds the navigator and navigates away from any missing file before the user tries to navigate to it.

Layer 2 (defensive): navigate() checks path.exists() before opening each candidate. This handles the race window between the deletion and the watcher event arriving, and also handles deletions on filesystems that do not support watching (where dir_watcher is None).

6.5 Extension-mismatch detection

decode_raster (ByExtension) reads 12 magic bytes via read_magic(), compares against the format implied by the extension, and returns Err on mismatch. load_image() catches the Err, calls sniff_format(), and returns WrongExtension. The image is displayed immediately. The rename dialog appears after the drain.

> ■ *image::open() and with_guessed_format() both sniff the header and succeed silently on misnamed files. The explicit magic check is the only reliable detection mechanism.*

6.6 offer_rename writability probe

Opening the file with write access before showing the dialog. On Windows all shell folders carry the read-only attribute by convention — checking it would suppress the dialog everywhere. Checking the file

itself is portable and accurate.

6.7 The two-WakeUp window reveal sequence

Window hidden → gray paint → decode thread → WakeUp → resize → pending_center = true → self-WakeUp → next WakeUp: center → set_visible → RedrawRequested paints. The self-WakeUp is essential for single-image archiver temp folders where no preload threads fire.

6.8 Immediate process exit

Background threads and the watcher thread hold senders. A graceful exit would wait for all to finish. std::process::exit(0) terminates immediately — correct for a stateless viewer.

7. Key Design Decisions & Trade-offs

notify-debouncer-mini for directory watching

The alternatives were: polling on a timer (no dependency, wastes CPU, has latency), checking path.exists() only at navigation time (fixes deletion errors but misses additions), or writing platform-specific ReadDirectoryChangesW/inotify code directly (large effort). notify-debouncer-mini is the standard solution, adds one dependency, and has zero steady-state CPU cost.

watched_dir field separate from navigator.current_dir

The navigator updates current_dir inside set_current(), which open_image() calls before the decode thread finishes. apply_image() therefore always sees the new directory already set — using it for the watcher change detection would mean the watcher is never installed. watched_dir is only updated inside watch_dir(), making the comparison in apply_image() reliable.

Two-layer external deletion safety

The watcher provides proactive rebuilding but has an inherent race window between deletion and the 50ms debounce firing. The path.exists() check in navigate() closes that window and also handles filesystems where watching is unavailable. Neither layer alone is sufficient.

Magic-byte validation in decode_raster

Every image-crate API that takes a path internally calls with_guessed_format(), which sniffs the header and silently succeeds regardless of extension. The explicit 12-byte read is the only reliable way to detect mismatches. The overhead is negligible compared to a full decode.

magic_to_format as single source of truth

Both sniff_format (wrong-extension recovery) and decode_raster (validation) delegate here, ensuring consistency. Adding a new format is a single-point change.

DecodeStrategy enum / Direction enum

Both replace bool parameters. decode_by_ext(..., DecodeStrategy::ByContent) and navigator.step(Direction::Forward) are self-documenting at every call site.

rebuild_current takes current_path parameter

The navigator's own state is stale by the time rebuild_current is called because set_current already ran. Passing current_path explicitly makes the method independent of when it is called relative to set_current.

DirEntry::file_type() over Path::is_file()

file_type() is free from the readdir() result. is_file() issues one extra stat() per file — 5000 extra disk hits on a cold cache for a large folder.

render() row-based optimisation

Filling entire margin rows with slice::fill avoids per-pixel branching. inv_scale precomputation replaces per-pixel division. composite() fast-paths for fully opaque and fully transparent pixels cover most real-world content.

offer_rename writability probe

Opening with write access is the only reliable portability-safe test. The parent directory read-only attribute is set by convention on all Windows shell folders.

8. Caveats & Known Limitations

No status bar

Filename shown in title bar; index and dimensions are not displayed.

Animated GIF only

Animated WebP or AVIF would need separate decoder paths.

JXL HDR clamped

f32_to_u8 clamps above 1.0. Standard JXL photos unaffected.

Animated JXL not supported

Only frame 0 is rendered. Opens without error as a still image.

Directory watching on network shares / virtual FS

watch_dir() fails silently if the OS watching API is unavailable or unsupported for the filesystem. Navigation still works; the sibling list is static. The path.exists() check in navigate() still protects against external deletions.

Work area heuristic on non-Windows

90%×85% of monitor size may not account for taskbars accurately.

SC_MAXIMIZE winit behaviour dependency

Relies on winit firing MouseInput for title-bar double-click. Observed behaviour, not a documented guarantee.

9. Installer — devoid_viewer_setup.iss

Inno Setup 6. Run iscc devoid_viewer_setup.iss after building the release binary. Output: Output\DevoidViewer_Setup.exe.

9.1 Why Inno Setup instead of WiX 3

WiX 3 evaluates Component Condition elements during the MSI costing phase before UI is shown, making per-user/per-machine scope switching impossible. Inno Setup handles it correctly via IsAdminInstallMode and HKA.

9.2 Installation scope

PrivilegesRequired=lowest + PrivilegesRequiredOverridesAllowed=dialog. Per-user default (no UAC); optional per-machine (UAC). HKA resolves automatically.

9.3 Supported extensions — 10 formats

jpg, jpeg, png, bmp, gif, webp, tiff, tif, ico, jxl. SVG removed.

9.4 File associations

Feature	Registry written	Notes
ProgId	HKA\Software\Classes\DevoidViewer.ImageFile	Always installed.
Open With	HKA\...\OpenWithProgIds	Adds to Open With without changing default.
Set as Default	HKLM\Software\Classes\.ext	Per-machine only. Avoids UserChoice.

9.5 The UserChoice problem

Writing only to HKLM avoids UserChoice entries. UserChoice has a cryptographic hash verified on every file open; stale entries survive uninstall and break associations.

9.6 Uninstaller

- taskkill closes running instances before file deletion.
- Folder removed via uninsalwaysuninstall.
- All UserChoice keys for 10 extensions deleted on uninstall.
- Shell icon cache refreshed via ie4uinit.exe.

10. Glossary of Rust Concepts Used

Ownership
Every value has exactly one owner. When the owner goes out of scope the value is dropped.

Borrowing
Lend a reference (&T;) without transferring ownership. References cannot outlive the value they point to.

Option
Either Some(value) or None. The safe replacement for null pointers.

Result
Either Ok(value) or Err(error). The ? operator propagates errors up the call stack.

Vec
A growable heap-allocated array. Used for pixel buffers, frame lists, and path lists.

HashMap
A hash table. O(1) average lookup. Used for the preload cache keyed by PathBuf.

Arc
Atomically Reference Counted smart pointer. Shared ownership across threads. Used for the Window handle.

Enum with data
Rust enums carry data per variant. Msg::Loaded carries a PathBuf, DecodedImage, and OpenMode.

then_some
(condition).then_some(value) returns Some(value) if true, None otherwise.

is_some_and
option.is_some_and(|v| pred(v)) — combines is_some() with a predicate check.

let-else
let Ok(x) = expr else { return; } — early return on failure without nesting.

Lifetime
A compile-time label on a reference. View<'a> borrows from App — cannot outlive App.

Closure
An anonymous function. move transfers ownership of captured variables into it.

match
Exhaustive pattern matching. Every enum variant must be handled.

debug_assert!
Panics in debug builds only. Catches state desync without release overhead.

#[derive(...)]
Auto-generates trait implementations: Clone, Debug, Default, etc.

Send
Marker trait: type is safe to transfer to another thread.

ControlFlow
Winit enum: Wait sleeps until any event; WaitUntil sleeps until a deadline.

EventLoopProxy
Thread-safe handle to the event loop. proxy.send_event() from any thread.

NonZeroU32
A u32 guaranteed non-zero at compile time. Required by softbuffer's resize.

cfg(windows)
Conditional compilation. Code compiled only on Windows.

unsafe
Opts into C API calls. Used for MessageBoxW, SystemParametersInfoW, PostMessageW.

type alias
type DirWatcher = ...; shorter name, no runtime cost.

Zero-size struct
struct WakeUp; occupies zero bytes. Used as a type-level event marker.

include_bytes!
Embeds a file's raw bytes into the binary at compile time.

process::exit(0)
Terminates immediately, bypassing Rust's drop sequence.

DecodeStrategy
Enum replacing a bool: ByExtension and ByContent are self-documenting.

Direction
Enum in nav.rs replacing a bool: Forward and Backward are self-documenting.

magic_to_format
Single source of truth for magic-byte format detection.

DirWatcher / watched_dir
Type alias and field for the filesystem watcher. watched_dir is tracked separately from the navigator to ensure the watcher is installed on the first image open.

DIR_WATCH_DEBOUNCE
Named constant for the 50ms watcher debounce window. Coalesces rapid filesystem events into a single navigator rebuild.